Design and Analysis of Algorithms

Module 5: Complexity Categories

This is an open educational resource.
Feel free to submit fixes, improvements, and new material here.

This week

This week we are going to talk about a very abstract topic: **complexity categories**.

A complexity category is a set of **problems** (not algorithms) that all share certain time bounds and characteristics.

Up til now, we have considered the bounds of *algorithms*. We have not yet considered that *problems* can be bounded too.

But why would we do this?

Why bound problems

Bounding problems is useful because it lets us describe the essential difficulty of solving that problem.

For example, general sorting is $\Omega(n)$. E.g., counting sort is $\Theta(n)$.
You cannot go lower than this and still see every element in the array.

Does that mean that it's worthless to try to find a $O(\lg n)$ sorting algorithm?

No, it means it's worthless to try to find a $O(\lg n)$ sorting algorithm that makes no assumptions at all (general sorting). There are situations where you might know that only a couple of indices of a skip list are out of place, and where they are.

Bounding problems helps us know when it's more productive to adjust our assumptions or desires than to try to solve a problem that may be impossible in the general case.

Grouping problems

It turns out, when bounding problems, we often group them into sets.

A very important set is P, which is the set of *decision* problems for which a deterministic turing machine that solves them in polynomial time is known.

What is a decision problem? Think of it as a problem that asks for a boolean solution.

All of these are decision problems, but not all have known solutions in P:

- Is this list sorted? (This one is in P)
- Is this a prime number? (In P as of 2002)
- Is this the shortest path through this graph? ($\in P$)
- Is this the *longest* simple path through this cyclic graph? ($\notin P$)
- Is this arbitrarily-sized sudoku solvable? ($\notin P$)

Motivation

Determining whether a list is sorted is $\Theta(n)$.
Multiplying two matrices together is $\Theta(n^3)$
(technically faster solutions are known, but the fastest is $\omega(n^2)$).

Why group these problems together? They seem like one is way faster than the other.

Let's revisit our basic machine model. We decided early on to use the RAM model of computation, where basically every memory or register access (and therefore, machine instruction in general) counts as 1 time unit.

Consider 3 different machines...

3 machines

1. A modern x64 desktop with terabytes of RAM, a bazillion core CPU, and a sick RTX video card that gets the high score on whatever 3D Max benchmark is current.
2. A Nintendo Entertainment System (NES)
3. A computer someone made in Minecraft.

I hope you will agree that these machines are different in their performance characteristics.

It's not just that one machine is faster than another (although it is absolutely the case that one machine is faster than the others). Basic operations have fundamentally different time bounds.

For example...

3 machines (2)

Consider multiplication. For any reasonable multiplication need (not considering BigInts) for most algorithms, the x64 desktop's `imul` or `mulsd` instruction suffices.

This instruction is $\Theta(1)$. It's contant time.

Again, technically we could say "but what if the number was really big", but say we only want to multiply numbers that fit in a machine register. It's $\Theta(1)$.

3 machines (3)

But the NES has a processor based on the classic 6502 CPU. This processor does not have a multiply instruction. If you want to multiply two integers, you have to code that with addition.

The naive approach to multiply $n \times m$ is:

```
int product = 0;
while (n > 0) {
    product += m;
    n--;
}
```

This naive approach is $\Theta(n)$.

3 machines (3)

A faster approach is to use an algorithm that applies the distributive property to binary numerals. It treats 7 * 7 as 7 * (1 + 2 + 4), adding a power of 2 for every 1 in the numeral.

```
int product = 0;
while (m > 0) {
    if (m & 1) product += n;
    n <<= 1;
    m >>= 1;
}
```

This algorithm is $\Theta(\lg m)$. Because it takes different amounts of time depending on how many bits are set in m. It's not constant. We can't do constant time multiplication on the NES.

3 machines (4)

Minecraft is a popular video game in which you can explore and build in a world made of blocks.

Some of these blocks, called redstone blocks, have special properties that allow them to turn lights and switches on and off, in response to other lights and switches.

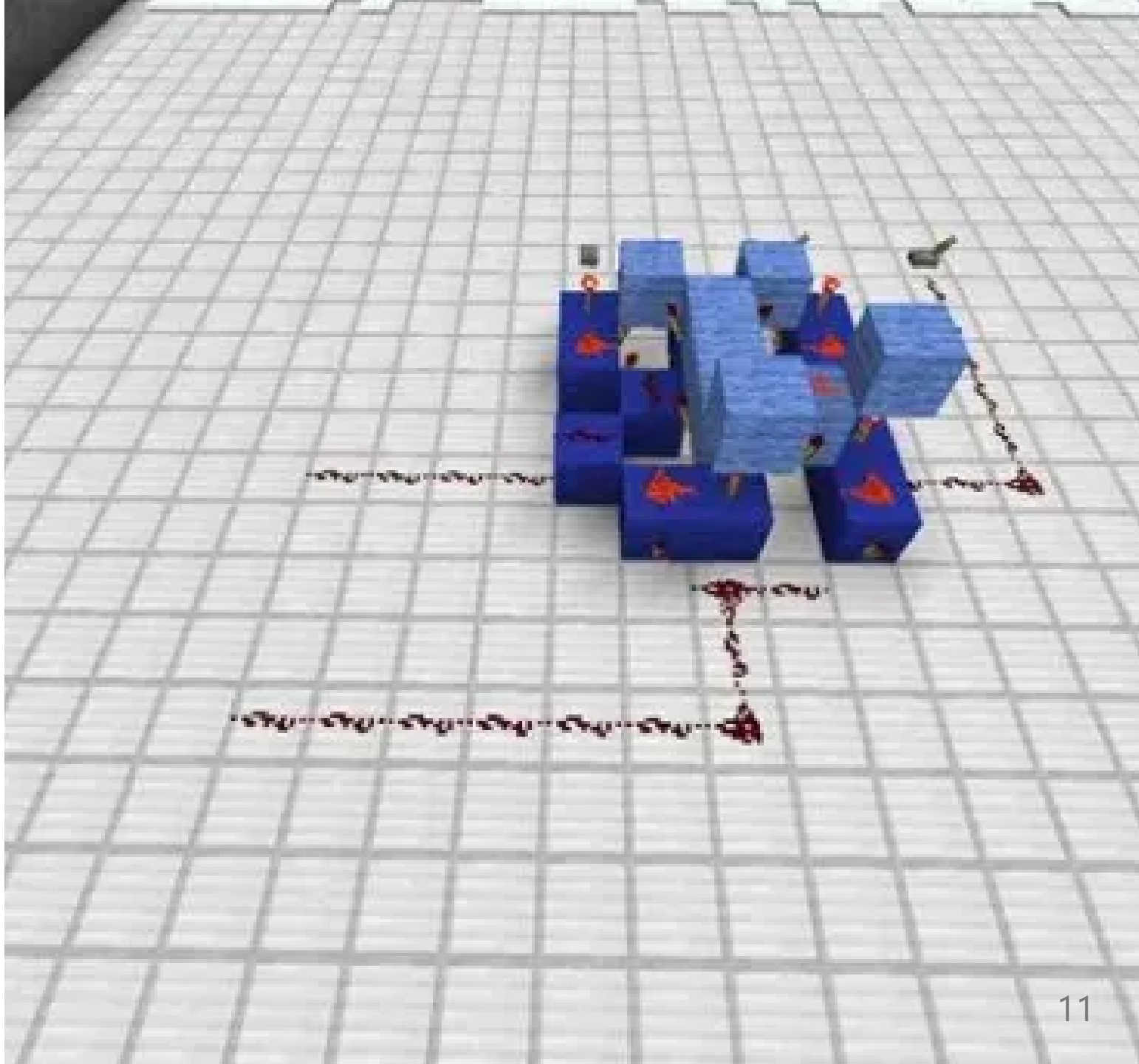
It turns out, the logic behind how this works is turing complete. Meaning that turing machines, and therefore programs, can be simulated inside this game.

3 machines (5)

However, using these blocks is tricky and slow.

The game looks at adjacent blocks and to slowly propagate the state forward.

As a result, adding even two bits is somewhat slow; adding an entire integer is orders of magnitude slower than doing it in an NES.



3 machines (6)

The NES can add reasonable-sized integers in $O(1)$, but multiplication is $O(\lg n)$.

Minecraft redstone computers cannot even perform addition quickly. Even adding two numbers is $O(\lg n)$, where n is the larger number. The time depends on the number of bits in that number.

If we apply the clever multiplication algorithm we saw before, it relies on bit shifts and additions, both of which are $O(\lg n)$ in minecraft. Since we are performing an addition of n and some shifts for each 1 in the m operand, multiplication is $O((\lg n)^2)$.

If we had used the naive multiplication algorithm, and naive addition by increment, multiplication would be $O(n^2)$

Computation in general

Changing from one kind of machine to another radically altered how long it took to do even basic things, let alone more complex operations.

However, one thing remained the same: multiplication takes a polynomial amount of time for all of these.

And therefore, the decision problem of "is this integer the result of multiplying these other two integers" can also be solved in polynomial time: do the multiplication and check the result.

Abstract computation

And we want to consider how long multiplication should take, but as a computer scientist. And a computer scientist should be able to work with many different media of computation:

- There are the digital electronic computers we know and love.
- There are analog electronic computers, which have been useful in the past and which people theorize may be useful for implementing neural networks.
- There are optical computers that use light for calculations
- There are chemical computers, where chemical reactions do computation.
- There are even biological computers (your brain may be/have one).
- Really, there are a whole bunch of computing paradigms.

Turing computation

But remember, turing machines are extremely simple. They are just collections of states and transitions guarded by the current state and the state of memory.

And in fact, there are many versions of the Turing machine. Some versions have RAM. Some verisons have variables. Some version have 2 tapes. Some versions have the tape as a circle. These have all been proven equivalent to one another.

So when we're talking about the difficulty of a problem, we really need to be abstract, because we could be solving that problem in all kinds of wacky media or turing machines with different capabilities.

And yet, we can still say something about it...

Class-P is robust across models of computation

A decision problem is in P if there is a deterministic turing machine (TM) which decides it in $O(n^k)$ time, for some constant k.

If you have an $O(n^k)$ algorithm on one reasonable model of TM, there is an equivalent algorithm that will run it in $O(n^{k+c})$ on another reasonable TM, for some constant c.

A TM model is *reasonable* if its state-changes and basic operations are simulable on a classical TM in polynomial time, and vice versa (no magic instructions).

It does not matter if you are using a classical Turing machine, a DNA computer, a redstone contraption, or an iPhone. Your polynomial time problem will have a polynomial time algorithm if you have a poly-time algorithm for any (reasonable) turing-equivalent computer.

Questions?

Classes other than P

Remember, P is the set of *decision problems* (problems with a yes/no answer) that can be solved in polynomial time on a deterministic turing machine, such as your computer.

There are many useful problems in P.

But there are many important problems we want the answer to for which there is no known solution that is in P.

What does that mean? There are at least three possibilities ...

Classes other than P (2)

1. There *would* be a P solution if we magically knew which action to take at every step. Equivalently, if someone gives us a solution, we can verify it polynomialy. If we had the ability to try every possibility non-deterministically, we could solve it in polynomial time. The set of these problems is called **NP**.
2. Some NP problems are harder than others. Some NP problems are so hard, that if we found a way to reformulate any of them into P problems, it would bust the paradigm wide open (all the exponential time NP problems would have polynomial time solutions). These special problems are in a subset of NP called **NP-Complete**.
3. Some problems are so hard, that they are at least as hard as **NP-Complete**, but can be even harder. *Sometimes even checking that a solution is correct is hard*, so even using a magic oracle we cannot obtain a solution in P time. We call the set of these kinds of problems **NP-Hard**

Put another way...

1. P is the set of problems with deterministic polynomial time solutions ($O(n^k)$)
2. NP is the set of problems with polynomial time *verification* of a solution, whether or not there is a known algorithm that solves it in $O(n^k)$. It turns out that every problem in NP is $O(2^{n^c})$ on a deterministic TM where c is a constant.
3. NP-complete is the set of the hardest problems in NP. We'll explain more why it's important to define this subset in a bit.
4. NP-hard is the set of problems that are at least as hard as NP-complete

Later, I'll show you a diagram of these sets. However, the reason I haven't done that yet, is it actually has to be two diagrams, because...

P = NP?

We don't actually know that P and NP are different sets. No one has proven it.

NP is *definitely* a superset of P. That is, if we have a polynomial time solution that runs on a TM, then we definitely have a polynomial time solution that runs on a non-deterministic TM (abbreviated NTM).

An NTM is just a TM that can magically predict what to do next. So if we're polynomial time *without* that magic power, giving it to us does not slow us down.

Therefore, all problems in P are also in NP. $P \subseteq NP$.

P = NP? (2)

However, we don't actually know whether all problems in NP are in P.

We can't really say that a problem in NP is *not* in P. Maybe we just haven't figured out how to solve it yet in polynomial time.

There might be a polynomial-time *reduction* from the problem to a problem in P.

Reduction means converting one problem into another, such that solving the other means solving the one.

Reduction

For example, I can *reduce* the problem of determining whether a given number is the product of two other numbers to the problem of determining if a matrix is the result of multiplying two other matrices. I just encode the two integers as if they were both 1×1 matrices, and apply my solution for matrices. Then I convert back.

I'm just re-encoding a simple data type twice, so this reduction takes $\Theta(1)$.

Conversely, the problem of testing a product of square integer matrices *can also be reduced* to the problem of multiplying and adding integers. For every $n \times n$ integer in the second matrix, multiply it by the n integers in each column of of the first one, and do some additions.

This particular reduction (there are others) takes longer: $O(n^3)$ multiplications need to happen. **But it is still polynomial time.**

Alchemy of reduction

You notice that our reduction of matrix multiplication to scalar multiplication is pretty much how we usually mutliply (small) matrices.

But sometimes we have a choice of different reductions. Example, my problem is that I want to know if an element is contained in a list?

- One reduction involves sorting the list first, typically $\Theta(n \lg n)$ for comparison sorts, and then solving the problem "is this element in this *sorted* list", which can be solved in $\Theta(\lg n)$.
- Another reduction involves building a hash-set from the list $\Theta(n)$ and then asking the question, "is this value contained in this hashset", with expected time $\Theta(1)$.

Alchemy of reduction (2)

As programmers, we have flexibility in which reductions we can perform.

There are a huge number of ways to re-imagine a problem. Doing so can give you more insight into both problems, but it can also offer a new solution.

So don't just think about what you're doing as solving a problem. Try to think about it as reducing one problem into another that you have a solution for. It's a kind of alchemy of problem solving.

Questions?

A little warning about problems

Before we continue, I feel like I should highlight something.

In this lecture we're specifically talking about P and NP, which contain *decision problems*.

Many of the problems we solve as programmers are not decision problems. For example, computing the product of two numbers is a function problem, not a decision problem.

But determining whether a given product actually *is* the product is a decision problem.

Practically computable function problems live in the sets FP and FNP.

Many of the theorems that apply to P and NP *do not* apply to FP and FNP. It's important to keep them straight! Keep me honest too, if I give a problem in the wrong set!

Reduction basics

To say that A reduces to problem B in polynomial time, on a deterministic TM, we write:

$$A \leq_m^p B$$

This is confusing, because if we *reduce* A to B, it sounds like A must be *bigger* than B, otherwise how could we reduce it? And yet we use $\leq$.

But really, this notation is about difficulty. If we can reduce A to B in polynomial time, then we know either that A is in the same class as B, or an easier class. For example, a polynomial is less than an exponential as n goes to infinity.

The p means "polynomial time". We don't care about reductions that are so slow we don't gain anything by doing them. The m means many to one (as opposed to many-to-many). It means that you can't generate lots of reductions and ask an oracle to choose the best one. We will create one instance of B for each instance of A.

NP completeness

For this class, assume that $A \leq B$ is equivalent to writing $A \leq_m^p B$ for problems A and B. We're going to assume that reductions are polynomial time and deterministic unless I say otherwise.

Suppose that the problem of checking scalar multiplication is called MUL.
And suppose that the problem of checking matrix multiplication is called MATMUL.

We saw that MUL $\leq$ MATMUL, and MATMUL $\leq$ MUL, implying that they are "equally" hard (and they are: they are both in P).

They aren't literally equally hard. Matrices take longer to multiply than scalars. But they are both polynomial time; they are in the same set.

NP completeness (2)

NP is an important category because it contains all the problems that we could theoretically solve in polynomial time if we were smart enough or had enough threads.

There is an interesting problem in NP called **SAT**.

This is the boolean satisfiability problem. It is the problem of, given some boolean formula like $(a \vee b) \wedge (\neg b \wedge c)$, is there some set of booleans that satisfies it? I.e., are there some true/false values we can fill in for a, b, or c that makes the expression true?

In this case, yes. a = true, b = false, c = true.

This is the only solution, but sometimes there are more than one, and sometimes there are zero. The problem is to determine if at least one solution exists.

NP completeness (3)

In 1971, computer scientist Stephen Cook had an important realization. He realized that *every problem in NP could be reduced to SAT*.

First, how do we know that SAT is in NP? Let's imagine you have some complicated boolean formula with n terms.

Determining if there is a solution is hard. We have to potentially consider every assignment of true or false to every variable, and there can be as many variables as terms.

In principle, this is $O(2^n)$, because there can be up to n variables, and there are 2 possibilities per variable.

NP completeness (4)

But, given a solution, determining if it is correct is very easy. We just fill in the trues and falses and evaluate all the operators. This takes linear time in the number of operands.

So on a non-deterministic turing machine, SAT is in NP. It has a polynomial time solution if we have the ability to try every combination at the same time.

Okay, SAT is in NP. But why would we care?

Everything in NP can be reduced to SAT

The Cook-Levin theorem, named after Stephen Cook and Leonid Levin, a Soviet (at the time) computer scientist who independently discovered the same things as Stephen Cook, states:

Any problem in NP can be many-to-one reduced to SAT in polyomial time

In other words, $\forall X \in \mathrm{NP}, X \leq \mathrm{SAT}$

Why? What's so special about boolean satisfiability?

The SAT reduction

Imagine we have a SAT solver. SAT solvers are not things that only exist in theory: people compete every year to make the fastest one.

We can give a boolean expression to our SAT solver, and it will tell us whether it has a solution or not. In practice, it will also tell us the solution, but to be strictly a decider, it is only required to output "yes" or "no".

Cook and Levin's goal was to convert *any* decision problem in NP into a SAT problem. For a problem to be in NP, there must exist a Turing Machine (deterministic or not) that solves it in polynomial time.

So we must assume that machine exists, and convert it into a SAT problem somehow.

The big idea

Given some problem X, our goal is to create a rule that converts each input of the problem into a giant boolean expression (in poly time) that answer three questions:

1. The machine runs correctly (it's not in two states at once or writing two symbols to one point at memory, etc.)
2. The machine actually halts
3. When the machine does halt, it answers "yes" (i.e., it accepts).

If the boolean expression we generate has a solution, then that means it is possible for the machine to run correctly, to halt, and to answer yes on that input. The problem was originally solved by a non-deterministic TM, so if it is capable of accepting the input, it *does* accept the input. So if we solved the original problem, this boolean expression will have a solution and vice versa.

The SAT reduction (2)

To determine whether the machine runs correctly, we literally encode a Turing Machine simulator as a boolean expression.

First, we can describe states of a TM. What do we need to know about a TM to simulate its next steps?

- We need to know its current state
- We need to know the state of every cell of the tape
- We need to know the location of the tape's head (which cell is "current")

We can encode these in boolean expressions. For example, we can have a list of variables $Q_{t,q}$, each of which is true only if the machine can be in state q at time t, and false otherwise. The number of states is constant, so there are $C \cdot p(n)$ of these variables in total.

The SAT reduction (3)

What about the tape? Turing machines have an input alphabet, which is the set of symbols that can be in any cell of the tape.

Define a giant list of boolean variables $S_{t,i,a}$, each of which means "at time t, index i on the tape has symbol a".

And also, define a list $H_{t,i}$, each of which means "at time t the head is over index i".

We add a boolean clause $\bigvee_{t=0}^{t=p(n)} Q_{t,q_a}$ for every accepting state q_a. So our boolean expression only has a solution if it is possible for the machine to be in an accepting state at some point.

All of these variables together define the set of states the machine *could* be in. So we can say "hey, if there is some way the machine *could* accept, then we say "yes" for this input.

The SAT reduction (4)

What if the machine is in two states at once? Or what if one tape cell has two different symbols in it? These conditions can't happen with turing machines, but they could happen with the boolean expressions we just introduced.

We need additional conditions that prevent the boolean expression from having a solution when the Turing Machine being simulated does not.

Enforce the following (using lots of "or-but-not" logic) for each (i, t):

- Exactly one $S_{t,i,a}$ is true for all a in the alphabet.
- Exactly one $Q_{t,q}$ is true for all q in the set of states.
- Exactly one $H_{t,i}$ is true. (the head is over exactly one cell)

But what about transitions?

The SAT reduction (5)

For every transition in the original machine, $q \overset{a}{\mapsto} (q', d)$, where q is the initial state, q' is the destination state, a is the symbol that needs to be under the head, and d is the direction to move the tape head, we add even more clauses to require:

$$(Q_{t,q} \wedge H_{t,i} \wedge S_{t,i,a}) \implies (Q_{t+1,q'} \wedge H_{t+1,i+d} \wedge S_{t+1,i,a'})$$

That is, for every state and transition, either one of those variables on the left is not set, or all of the variables on the right are set.

So any time we *are* in the state on the left, it has to be the case that the machine at the next time step is in the correct state, too.

The SAT reduction (6)

The head must be over exactly one cell at a given time t.

So it must be over at *most* one cell...:
for all $i, i', i \neq i', \neg H_{t,i} \vee \neg H_{t,i'}$

But also at *least* one cell:
$\bigvee_{-p(n) \leq i \leq p(n)} H_{t,i}$

Note the bounds on i. We could start going left or right with every transition, and since we take $p(n)$ steps, that's as far to the left or right that the head could be.

Similary, we can only be in one state at a time.

The SAT reduction (7)

The tape can only be changed at the head:

$$\forall a, a', a \neq a', S_{t,i,a} \wedge S_{t+1,i,a'} \implies H_{t,i}$$

In other words, if the tape at position i changes from a to a' at time t, then the head needs to be there at time t.

The SAT reduction (8)

The machine needs to start out in the right state.

$S_{0,i,a}$ is true if the initial input has symbol 'a' at position 'i'.

Also, we need to start in the right state: Q_{0,q_s}, where q_s is the starting state.

Finally, the tape needs to start at position 0: $H_{0,0}$

The final problem

As a summary from here, we have this giant boolean expression, consisting of all the following anded together:

- $S_{0,i,a}$ for all i and a, meaning that we start in a given state
- Q_{0,q_s} we start in state q at time 0
- $H_{0,0}$ the head starts at position 0 at time 0.
- for all $i, a, a', a \neq a'$, $\neg S_{t,i,a} \vee S_{t,i,a'}$, at most one symbol per tape cell
- $\bigvee_{a \in \Sigma} S_{t,i,a'}$ at *least* one symbol per tape cell
- $\forall a, a', a \neq a', S_{t,i,a} \wedge S_{t+1,i,a'} \implies H_{t,i}$, tape only written at head

and...

The final problem (2)

- $\forall q, q', q \neq q', \neg Q_{t,q} \vee \neg Q_{t,q'}$, we're in at *most* one state
- $\vee_q Q_{t,q}$, we're in at *least* one state.
- for all $i, i', i \neq i', \neg H_{t,i} \vee \neg H_{t,i'}$, the head is over at *most* one cell
- $\vee_{-p(n) \leq i \leq p(n)} H_{t,i}$, and at *least* one cell
- $(Q_{t,q} \wedge H_{t,i} \wedge S_{t,i,a}) \implies (Q_{t+1,q'} \wedge H_{t+1,i+d} \wedge S_{t+1,i,a'})$, transitions respected
- $\vee_{0 \leq t \leq p(n)} \vee_{q_a} Q_{t,q_a}$, must be in an accepting state at some valid point in time

The final problem (3)

If that giant, massive boolean expression has a solution, then it means that the turing machine we're simulating has the possibility of reaching an accepting state.

Which means it *does* reach an accepting state (because NTMs explore every possibility)

Which means we can answer the original problem, but using the solution for SAT instead of the turing machine we had originally.

This is a reduction, and it's one of the most important ones. It's a reduction that works for any problem in the entire, massive set of NP.

That is, if we have a turing machine that solves a problem in NP, we can now convert it to a SAT problem in polynomial time without an oracle and solve it with a SAT solver. Therefore, if anyone makes a SAT solver in P, we have a guaranteed P time solution.

Questions ?

Practice problems

All of these problems suppose that we've performed the reduction from X to SAT.

1. If we know $H_{t,i}$, what do we know about $H_{t+1,i'}$?
2. If we know that $S_{t,i,a}$, what can we say about $S_{t,i,b}$, where $a \neq b$? Are any of those true?
3. If the original Turing machine had no accepting states, what can we say about the number of solutions to the boolean expression we create?
4. Suppose the following $Q_{t,q}$ terms are true: $Q_{0,1}$, $Q_{1,2}$, $Q_{2,3}$, $Q_{1,4}$
   Why is this set of terms invalid?

Practice answers

1. We know $H_{t+1,i-1} \vee H_{t+1,i} \vee H_{t+1,i+1}$, because the head can only move one cell per time unit.
2. No. If $S_{t,i,a}$, then $\neg S_{t,i,b}$ for all $b \neq a$. Otherwise, we could have more than one symbol written to the same tape cell.
3. There won't be any. The answer will be "no", just like it was for the original TM.
4. At time $t = 1$, we are in two states, 2 and 4.

How long does it take?

That's a ton of work.

Imagine we have a simple problem like "is this list sorted", and we convert it into that boolean monstrosity so we can run it on a SAT solver. Is this transformation really polynomial time?

Yes. Building the table takes, at worst, $O((p(n))^3)$, because:
- There can't be more than $p(n)$ time units, so t is bounded.
- Even if there were more than $p(n)$ alphabet symbols, we wouldn't have time to write them all in $p(n)$, so take a subset of size $p(n)$.
- We can only write to, at most, $p(n)$ tape cells.

How long does it take? (2)

Therefore, creating that giant table of tape states $S_{t,i,a}$ is cubic in $p(n)$. It might require us to write up to $p(n) \times p(n) \times p(n)$ variables.

And if $p(n)$ is a polynomial, $(p(n))^3$ is a polynomial. This is the most expensive step, and it dominates the others.

However, it's a little slower even, because we need to run this on a Turing Machine. We often use the bit model for runtime bounds on classical Turing Machines, and storing the time t and index i is usually done in binary. This requires a logarithmic number of bits.

So the final runtime on a turing machine is bounded by $O(\lg(p(n))(p(n))^3)$

How long does it take? (3)

But maybe you're curious how long SAT takes?

Modern SAT solvers have all kinds of clever heuristics and multi-threading capabilities. Many package managers, like Conda and Apt, (but not Pacman, sorry Arch users), are basically just wrappers around a SAT solver.

Most of the time, it's possible to make these solvers run quickly. But, fundamentally, there are still 2^n combinations of variables to consider (with each representing whether a particular package and version is included). That's why sometimes Pip can take 15 minutes and then crash: it got stuck in a bad SAT solve.

For this reason, some dependency managers just say "we will only use the latest version". And you get Arch + pacman. In the worst case, SAT takes $\Theta(2^n)$

Why would anyone do this?

"Would anyone really do this?" No. If you have an algorithm that tells you whether a list is sorted, and you want to *actually use it*, to, you know, see if a list is sorted, you would never encode it as a SAT problem and run a SAT solver.

It goes from being $\Theta(n)$ to $O(2^n)$, where n is a way bigger number. Awful. Horrible.

So what was the point?

P = NP? (2)

By showing that every problem could be reduced to SAT in polynomial time, Cook and Levin did something very interesting.

Imagine that we go the other way around, and reduce SAT to something else in polynomial time?

Now we have proven that SAT and the other problem are "equally-ish" hard.

So what if we somehow proved that SAT could be reduced (in polynomial time) to the problem of checking if two matrices multiplied gave a particular product? ...

P = NP? (3)

...Then we would have a poly time solution on a *deterministic* turing machine for SAT.

That is, we would have proved that SAT is actually in P.

And you know what? Everything in NP can be reduced to SAT...

Which means that everything in NP can be reduced to the problem of checking a matrix product (by way of SAT)...

Which means that everything in NP has a polynomial time algorithm that runs on a deterministic turing machine...

Which means that everything in NP is actually in P!

NP-complete

In fact, SAT has been reduced to many, many other problems. But none of them is in P.

The set of problems that is equally as hard as SAT is called "NP-complete". It is the set of the hardest problems in NP.

P = NP? (4)

We already know that P $\subseteq$ NP

If you could reduce SAT, or any NP-complete problem, to something in P, you would prove that NP $\subseteq$ P.

And this would prove that P = NP.

And you would win a million dollars.

You would also win a million dollars if you proved that P $\neq$ NP, which seems much harder (you'd have to show that there is no possible reduction from any np-complete problem to any problem in P in poly time)

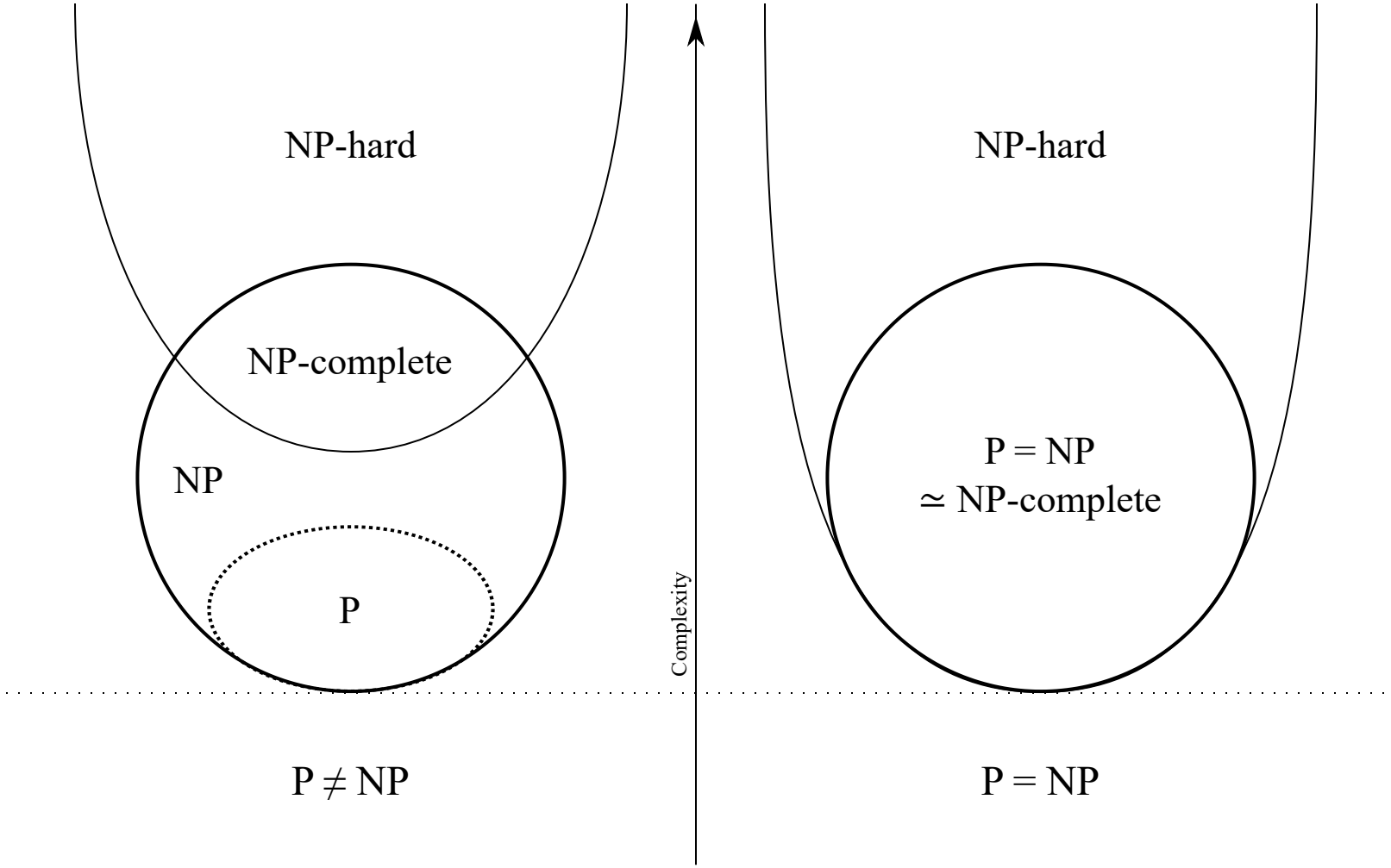
P = NP? (5)

According to polls, most (about 80%) of computer science researchers believe P $\neq$ NP.

It makes sense. It's hard to imagine how you could convert SAT into something that runs in polynomial time.

However, there's a weird possibility. Donald Knuth (question 17) believes actually that P = NP. He feels that there is space for polynomial algorithms with enormous exponents that do rote operations on all the bits of a problem. However, he feels that the proof will likely be non-constructive. Meaning, it will be an indirect proof, and it won't come with an algorithm that tells you how to convert an NP problem to a P problem.

So you would know your RSA encryption isn't quite as secure as you thought, but wouldn't know for sure if the algorithm that breaks it is practical. Great.



(Image by Behnam Esfahbod, CC-BY-SA 3.0, 2007-11-01)

(NP-hard is just everything at least as hard as NP-complete)

Why don't we care as much about space?

One last thing: why so much about bounding time and not bounding space?

Because so far, we've focused on polynomial time algorithms in this course. And an algorithm that uses polynomial time also uses polynomial space.

If writing to ram takes 1 operation, and you perform $p(n)$ operations total, then you can write to at most $p(n)$ cells of ram.

This is generally why we focus more on time bounds than space bounds.

However, in the latter half of this course, we'll see some examples of NP-complete problems that can be sped up a lot by using a bunch of space. So it does still matter.

PSpace

A program can't use more space than time. However, it can use much more time than space.

PSpace is the set of problems that take a polynomial amount of space. It actually includes some problems that are harder than the hardest problems in NP.

More practice

These are some practice problems based on popular games. If you don't know the rules for one of these games, try to come up with a question about a game you are familiar with, and encode it as a SAT problem.

1. Suppose you wanted to solve the decision problem "this sudoku has a solution". How could you express that as a SAT problem?
2. Suppose you were looking at a chess board and you wanted to know whether the game was over (in checkmate), how could you express that as a SAT input? No need to be too precise.
3. Suppose you wanted to know if a given wordle game could be won with a given word list, based on the highlighted tiles you've seen on the last guess. How would you encode that?

Questions?

Last Class

Last class we learned a ton:

- The importance of complexity categories: different computers have different capabilities, but polytime is polytime.
- The complexity categories P, NP, and NP-complete
- Why SAT is so important
- How to reduce an NP problem to SAT

This class

But there's one thing missing...

Remember how we said that any problem in NP can be reduced to SAT, but what about reducing SAT to other problems?

There is an entire class of problems in NP that SAT has been reduced to. These are called the NP-complete problems.

Many of these reductions are creative, and it helps to understand them. Let's take a look!

SAT $\rightarrow$ 3SAT

SAT feels like a very open-ended problem. Given any legal expression composed of boolean variables and boolean operators, determine whether it has a solution.

It turns out, this problem is actually equivalent to a problem that seems much simpler: 3SAT

The problem statement is like this:
Given a boolean expression in 3CNF, does it have a solution?

But what is 3CNF?

3-CNF

3-CNF stands for "three term conjunctive normal form".

Conjunctive normal form means that all of the conjunctions (ands) appear at only the highest level. That is, they happen last when evaluating.

$A \wedge (\neg B \vee C) \wedge (D \vee F \vee G \vee H)$ is in conjunctive normal form
$A \wedge (\neg B \vee (C \wedge D)) \wedge (F \vee G \vee H)$ is not, because of the $C \wedge D$ hapenning early.

3-CNF (2)

3-CNF is when we follow the rules for CNF, but also each term is the disjunction (or) of 3 terms, like this:
$(A \vee B \vee C) \wedge (D \vee \neg E \vee F) \wedge (G \vee H \vee I) \wedge \cdots$

Every boolean expression can be converted to CNF. The main trick is to replace every operator (such as xor or nand) with its equivalents using only and, or, and not. And then, to apply DeMorgan's laws. And it turns out, any CNF expression can be 3CNF.

SAT $\rightarrow$ 3SAT (2)

It might surprise you that the general problem of SAT can be reduced to the seemingly simpler problem of 3SAT. But it can: we just have to add more terms.

Given an arbitrary boolean expression composed of and, or, and not, we can convert it into a 3SAT expression by following some rules. We do it clause by clause

First, the SAT clause might already be in 3SAT: $A \vee B \vee C$. In this case, we're done.

What if it only has 1 term? Then $A \equiv A \vee A \vee A$
And 2 terms? $A \vee B \equiv A \vee B \vee B$

Okay, so for 3 or fewer terms, it's easy to turn a SAT clause into 3SAT clause. But what do we do for 4 or more terms?

SAT $\rightarrow$ 3SAT (3)

We're assuming that we're already in CNF, so each clause (the ones being anded) looks like this: $C_i = (\ell_0 \vee \ell_1 \vee \ell_2 \vee \ell_3 \vee \cdots \vee \ell_k)$

We can convert this to a number of 3CNF clauses by adding more variables like this: $C_i = (\ell_0 \vee \ell_1 \vee y_0) \wedge (\neg y_0 \vee \ell_2 \vee y_1) \wedge (\neg y_1 \vee \ell_3 \vee y_2) \wedge \cdots \wedge (\neg y_{k-3} \vee \ell_{k-1} \vee \ell_k)$

The "y" variables.

These y variables are just added to the formula, but they don't change whether or not there is a solution. Think of y_0 as meaning "there's a true clause later". For example, if y_0 is true, then $(\neg y_0 \vee \ell_2 \vee y_1)$ means that ℓ_2 is true or there's a true value later (y_1).

Another way of seeing $(\neg y_0 \vee \ell_2 \vee y_1)$, is that it's saying "or maybe there *isn't* a true value later, and actually it's ℓ_2 that is the true one. But if not, then there's a separate true variable in a future clause (y_1)

If there *isn't* a true clause later and if ℓ_0 and ℓ_1 are both false, then the whole original clause would have been false. Likewise, if there is a clause later that is true, only one of the original clause variables could satisfy it, because the

SAT $\longrightarrow$ 3SAT (4)

If this is confusing consider this: the original clause was a giant "or" of lots of variables.

A single one of them being true makes the whole clause true.

Therefore, if, say ℓ_1 is true, we can now set all the y variables to false, and the whole thing will be true.

If ℓ_0 and ℓ_1 are false, then we need one of the other ones to be true. So $y_0 = \text{true}$ forces l_2 to be true (because its clause has $\neg y_0$), or else, it forces l_3 to be true (because of $\neg y_1$), etc.

A solution to the 3SAT clause implies a solution to the original clause, and that is true for all the clauses in the expression.

SAT $\rightarrow$ 3SAT (5)

How fast is this reduction?

It should be reasonable that it is a polynomial time reduction. We iterate over every clause in the original expression. We iterate over every term in every clause. We construct at most one new clause per term. Therefore, if n is the number of terms, this reduction is $\Theta(n)$.

It should also be reasonable that 3SAT is a member of NP. We could try every combination of truth-value assignments on a non-deterministic machine and verify the results in polytime, the same as we did with SAT.

We know that NP $\leq$ SAT by Cook-Levin, and now we know SAT $\leq$ 3SAT by the reduction we just did, so we know 3SAT is in NP-hard. 3SAT $\leq$ SAT is trivial (a 3SAT formula is already a SAT formula). Therefore 3-SAT is NP-complete.

Reasoning about reductions

Any problem that is both NP-hard and NP is NP-complete. SAT $\leq$ 3SAT proves the hardness of 3SAT, 3SAT $\leq$ SAT, means that 3SAT and SAT are in the same category. NP-complete is the name of that category.

If problem A can be reduced in polynomial time to problem B, then A is "less or equally hard" than B: $A \leq B$

If problem B can be reduced in polynomial time to problem A, same thing: $B \leq A$

If they both apply, then A and B are roughly equally hard. They are in the same class.

Let's do a quick knowledge check...

Knowledge check

1. Suppose we reduce a problem to SAT. What does that tell us about the problem?
2. Suppose we then reduce SAT to that problem? What do we know then? Consider the two cases that the problem is in P, or that it is in NP but not P (NP — P).
3. Give an example of a reduction that would prove P = NP.

KC answers

1. It tells us the problem is NP. Because A $\leq$ SAT and SAT $\leq$ NP.
2. If the problem is in P, it means P = NP. If the problem is NP $-$ P, it means the problem is NP-complete.
3. Reducing 3SAT to the problem of determining if a list is sorted would prove P = NP.

Questions?

The clique problem

In everyday language, a clique is a group of friends who behave exclusively, keeping other people from joining their group.

In computer science, a clique of size K is a set of K vertices in a graph that are all connected to each other.

The image to the right shows a clique of size 3. 5, 2, and 1 are all adjacent. This is called a 3-clique. Can you find some cliques of size 2?

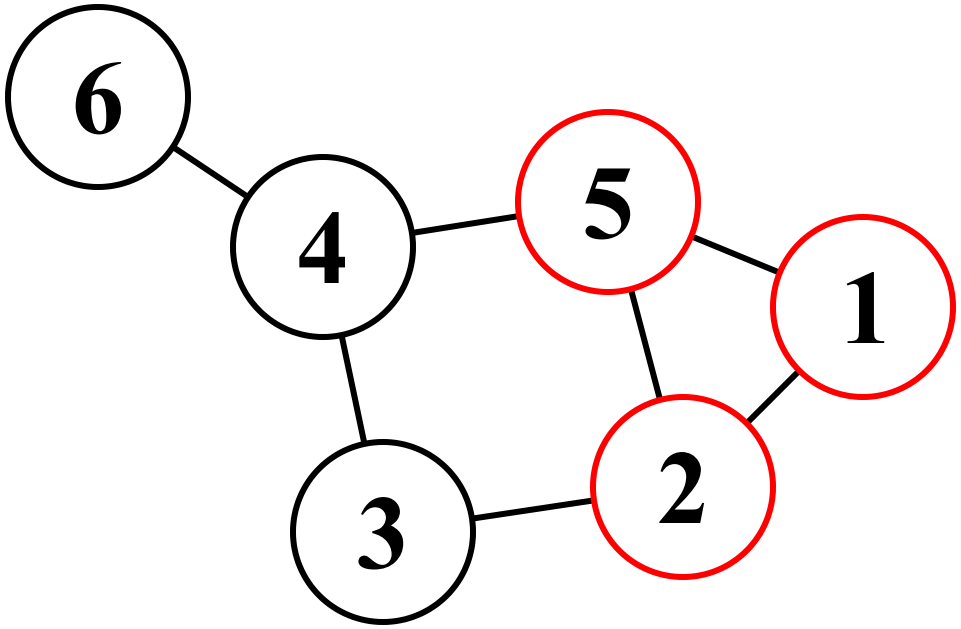


Image by David Remahl, public domain.

The clique problem (2)

This is an NP problem. Finding whether there is a clique of size 10 in a graph can be brute-forced by considering every combination of 10 nodes and checking to see if they are adjacent.

Checking a solution is easy, just check to see if there are enough vertices given, and that they are connected. This is clearly polynomial, so this is an NP problem.

If it's a problem in NP, we know that it can be reduced to SAT. It turns out, this problem is NP-complete. How do we prove that?

After proving that it's in NP, we need to reduce an NP-complete problem to clique in polynomial time. If we can do that, clique is NP complete.

SAT $\leq$ CLIQUE

Shortly after Stephen Cook's seminal paper, Richard Karp took 21 NP problems and reduced SAT to them (proving NP-completeness). One of these was SAT $\rightarrow$ CLIQUE.

It worked like this: create a vertex for every pair of (v, c), where v is a literal (i.e., a variable or a negation of a variable), and c is a clause that v appears in.

So if we had this formula: $(X \vee Y) \wedge (Y \vee \neg Z) \wedge (Z \vee X)$, define these:
$(X, X \vee Y), (X, Z \vee X), (Y, X \vee Y), (Y, Y \vee \neg Z), (\neg Z, Y \vee \neg Z), (Z, Z \vee X)$

Note, each of these is just a point. We haven't connected them yet.

$(X, X \vee Y)$ represents the statement "X is true, making $X \vee Y$ true".

This is how we connect them...

SAT $\leq$ CLIQUE assignments

Connect any nodes where the clauses are **different** and the variable assignment is not contradictory. That is, for two points, if their clauses are different but the variables aren't A and $\neg A$, connect them. Connections:

- $(X, X \vee Y)$ is connected to every other node except $(Y, X \vee Y)$.
- $(X, Z \vee X)$ is connected to every other node except $(Z, Z \vee X)$.
- $(Y, X \vee Y)$ and $(Y, Y \vee \neg Z)$ are connected to every other node, except for nodes with the same clause
- $(\neg Z, Y \vee \neg Z)$ and $(Z, Z \vee X)$ are connected to every other node except nodes of the same clause as well as each other, because $\neg Z$ is not compatible with Z.

[Let's whiteboard/mermaid it if time permits]

SAT $\leq$ CLIQUE assignments

We want at least a 3-clique: there are 3 clauses in the original expression, and a 3-clique would mean they could all be simultaneously true. For k clauses we need a k-clique.

I.e., a clique here represents clauses that can all be true at the same time, because none of them in the clique have contradictory variable assignments (e.g., Z and $\neg Z$)

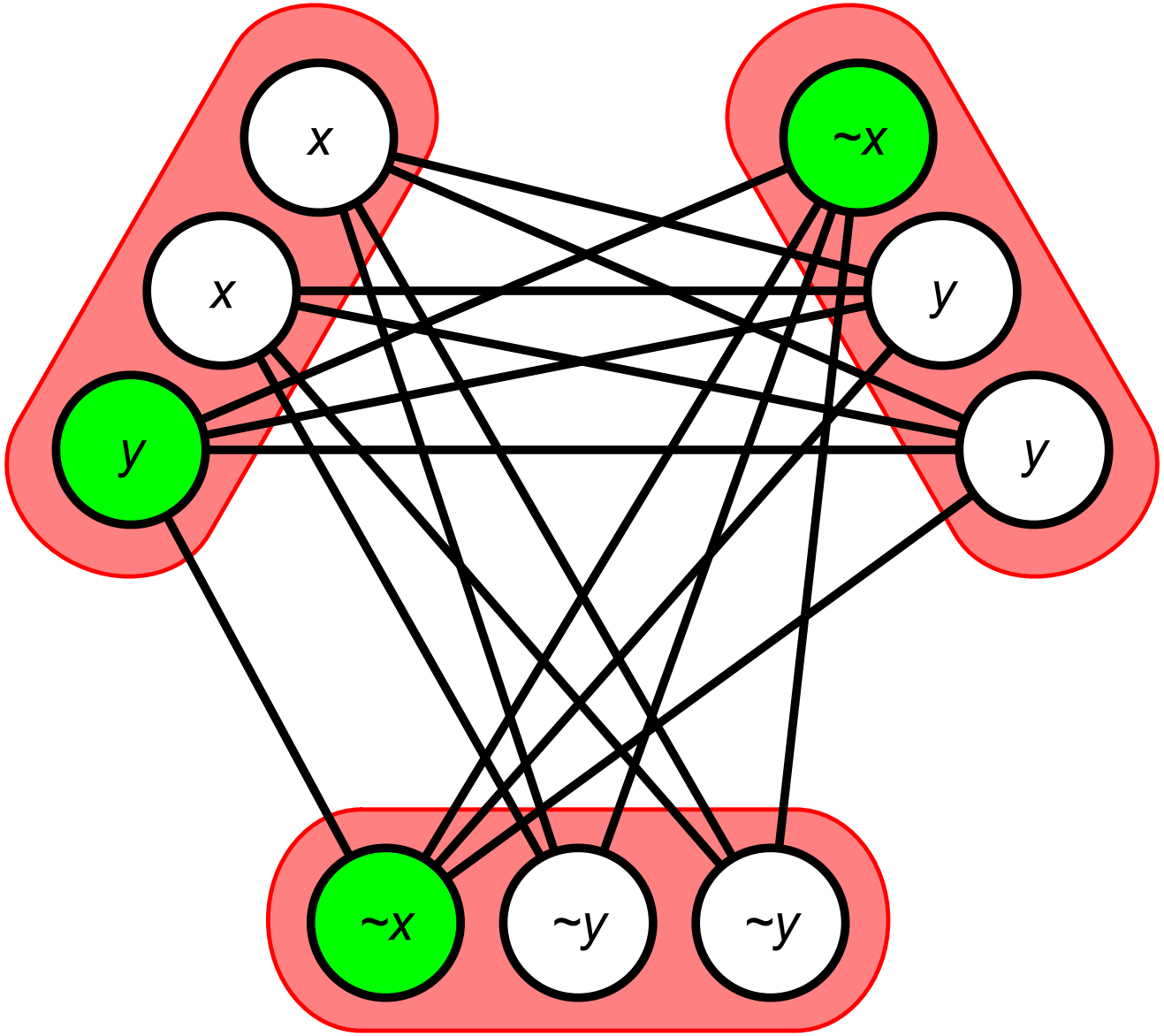
In our case, we have more than one to choose from. One of them is:
$(\neg Z, Y \vee \neg Z), (Y, X \vee Y), (X, Z \vee X)$

This means it is possible to satisfy the equation $(X \vee Y) \wedge (Z \vee X) \wedge (Y \vee \neg Z)$ using the assignments, Y = true, X = true, Z = false. (it's fine that $Y \vee \neg Z$ is connected to $Z \vee X$, it's only the assignments that can't be contradictory)

In general, any clique of size k has to have k different clauses in it, so we found a way to satisfy our 3 clauses without a single pair of contradictions. In this case: more exist.

Another example

This is a subgraph of the cliques for the boolean expression $(x \vee x \vee y) \wedge (\neg x \vee \neg y \vee \neg y) \wedge (\neg x \vee y \vee y)$. The chosen assignments are highlighted in green.



Practice problems

- Create a random boolean expression with at least 4 clauses and 4 different variables, with 3 variables or their negations per clause.
- Construct the graph according to the SAT $\rightarrow$ CLIQUE reduction.
- Does your expression have a solution?
- If so, modify it so that it does not. If not, modify it so that it does. Notice how it either creates or breaks cliques.

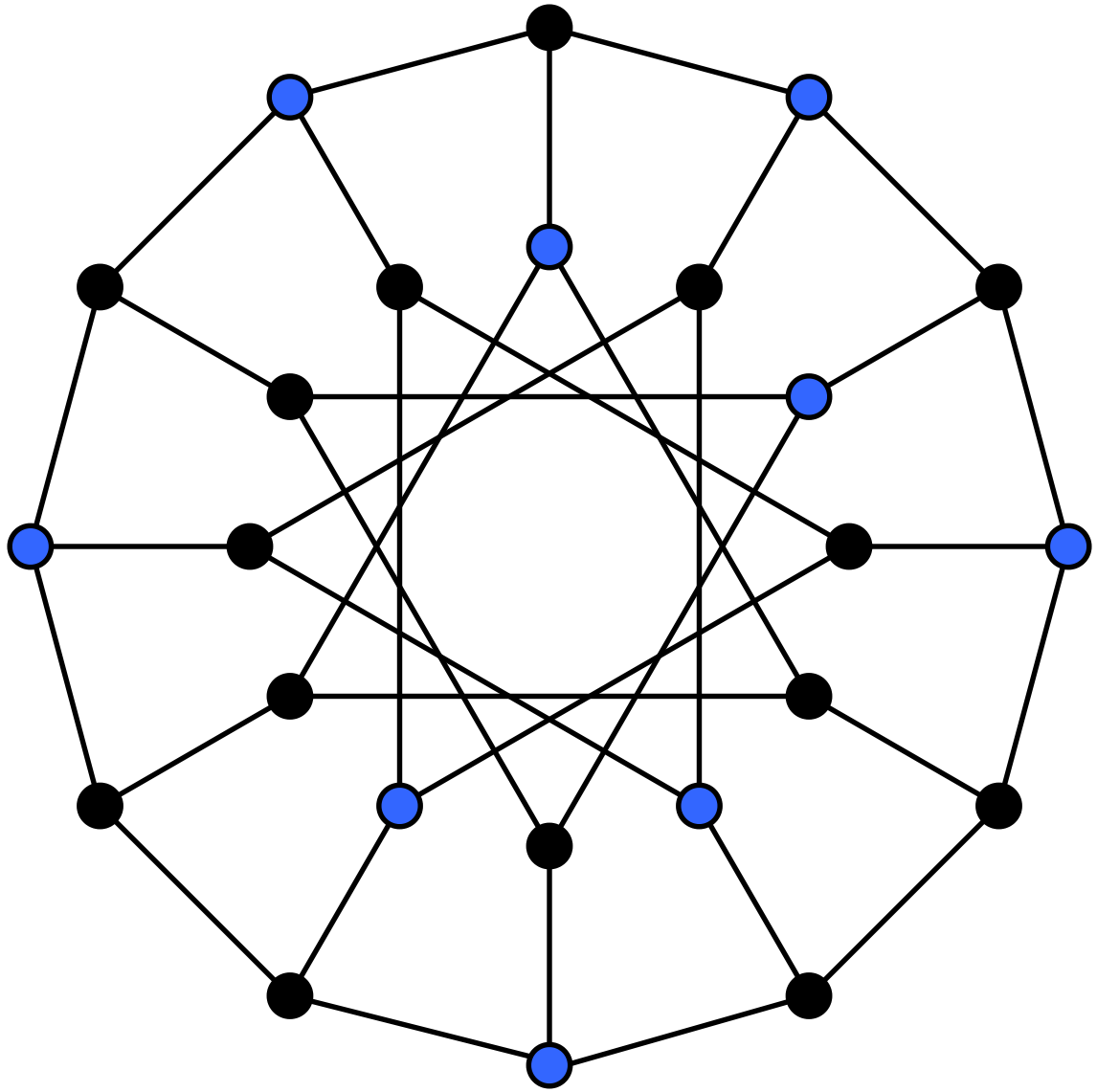
Questions?

INDSET

Now that we understand the clique problem, this one will be easy.

INDSET stands for "independent set". k vertices form an indset if *none* of them are adjacent. It's like an anti-clique.

In the image, the blue vertices form a 9-indset.



Reduce INDSET $\leq$ CLIQUE and CLIQUE $\leq$ INDSET

Do it for me.

How can I reduce these problems to each other?

INDSET $\leq$ CLIQUE

For this reduction, just invert all the adjacencies. So if nodes A and B are connected make them unconnected and vice versa.

Now, if we had a k-indset (none of the nodes were connected), we now have a k-clique (all of them will be connected). Therefore, our k-clique solver will return true for our inverted graph if and only iff our original graph had an indset of size k.

CLIQUE $\leq$ INDSET

This is identical. Invert the adjacencies. If we had a clique before, we have an indset now of the same size.

CLIQUE $\leq$ INDSET $\leq$ CLIQUE

Are these reductions polynomial time?

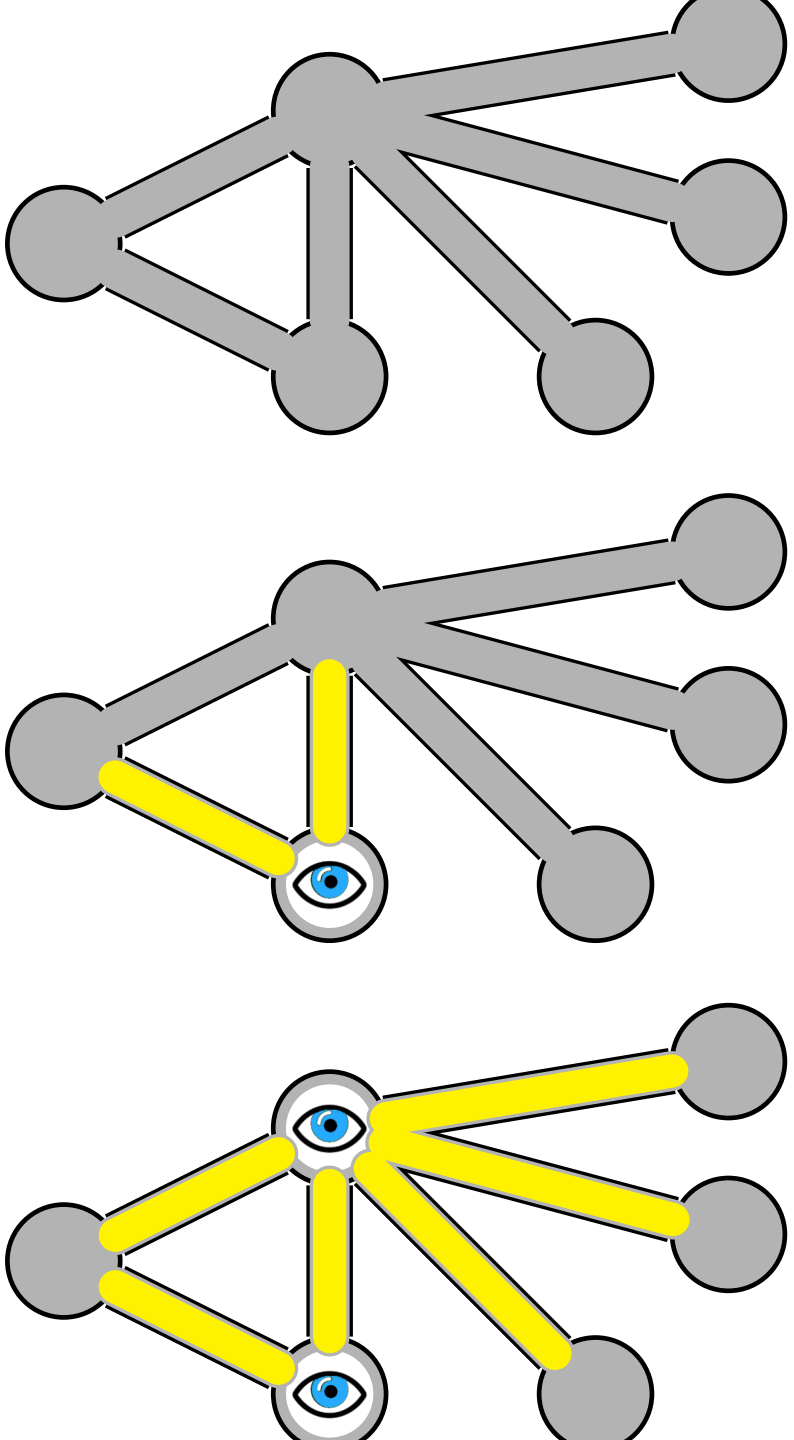
Yes. If there are n nodes in a graph, there are necessarily $O(n^2)$ connections between them. Therefore, inverting all the connections is $O(n^2)$.

Vertex Cover (VC) problem

A k-vertex cover occurs when there are k nodes such that every edge in the graph is connected to at least one of the k nodes.

For example, in the bottom image to the right, there is a vertex cover of size 2.

There is a functional version of this problem to find the *minimum* vertex cover, but we are concerned with a decision problem: *is* there a vertex cover of size k? In this case, yes for ≥ 2, no for 1.

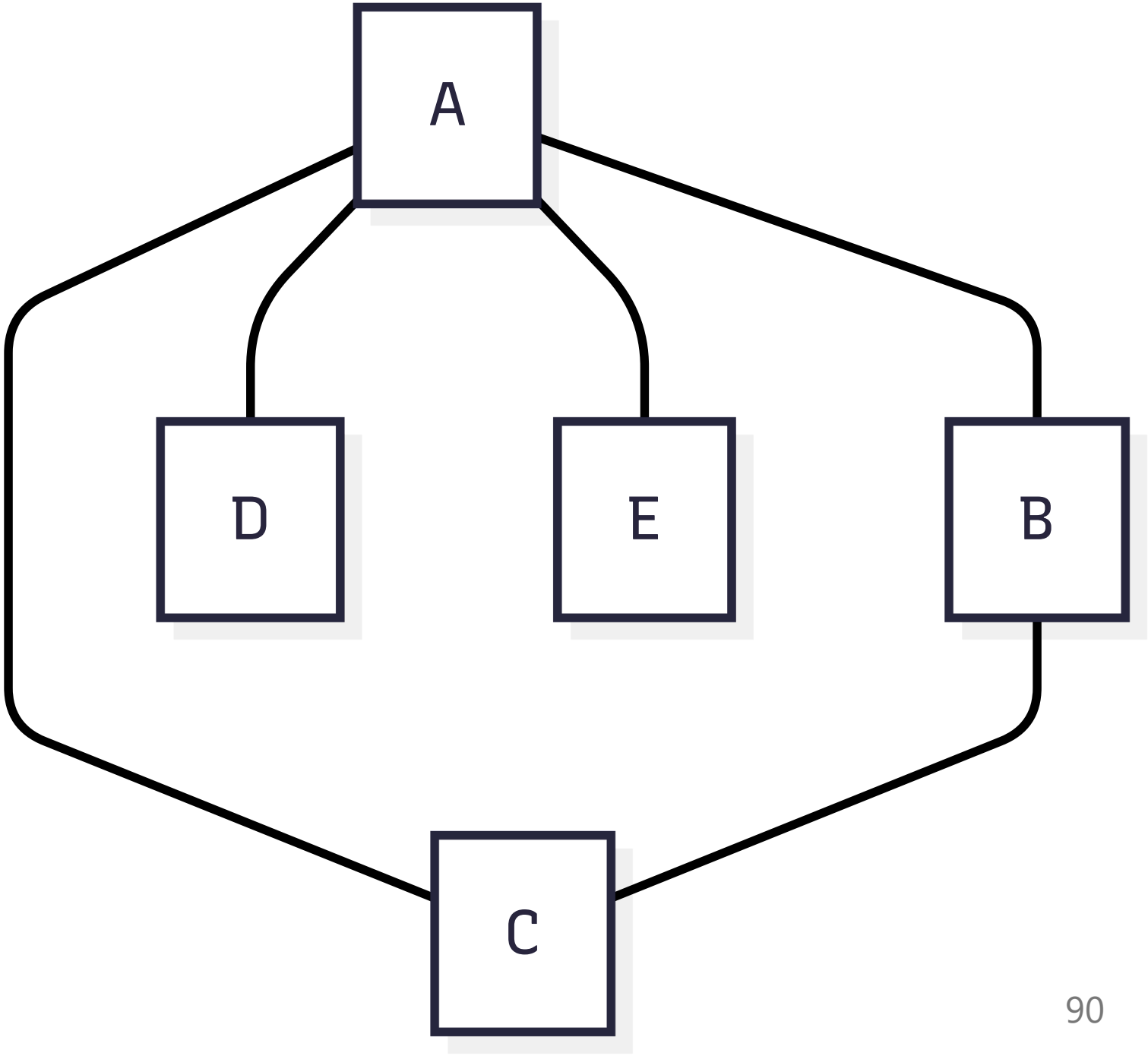


VC $\leq$ INDSET $\leq$ VC

It turns out, if there are n nodes and there is a clique of size k, then there is an indset of size $n - k$

Consider the graph on the right. There are vertex covers of size 2. One of them is $\{A, B\}$

Is there an indset of size 3?



VC $\leq$ INDSET $\leq$ VC (2)

Yes, $\{D, E, C\}$ is an indset of size 3.

It turns out, we don't even need to modify the graph. Knowing that there is a VC tells us there is an indset and vice versa.

Why does this work? Imagine that you have an indset. The nodes are not adjacent to each other. If you take the compliment (i.e., the nodes not in the indset), it would include all the adjacent nodes and therefore all the adjacent edges.

If the graph is connected, it would have to include all the gaps between the nodes in the indset. If the graph is not connected, then there are random disconnected nodes that do not have edges to connect.

The same argument works in reverse. If I have a vertex cover, then its compliment includes nodes that are adjacent to the cover but not touching each other.

Be careful

Just because there *is* an indset of size 3 doesn't mean that it's the largest indset.

Suppose we describe a graph in which nodes A, B, and C are all connected to each other in a clique, but there is also a connection from C to D.

This graph has a 3-clique, and it also has indsets of size $4 - 3 = 1$, as we would expect.

But it *also* has indsets of size 2!

Questions?

Knowledge check (2)

- We've shown VC $\leq$ INDSET $\leq$ VC
- We've shown CLIQUE $\leq$ INDSET $\leq$ CLIQUE
- What can we say about the relationship INDSET and VC?

KC 2 answers

INDSET and VC are in the same class. They are both NP-complete.

Because VC $\leq$ INDSET $\leq$ VC and CLIQUE $\leq$ INDSET $\leq$ CLIQUE, then
VC $\leq$ INDSET $\leq$ CLIQUE and CLIQUE $\leq$ INDSET $\leq$ VC

By transitivity, VC, INDSET, and CLIQUE are all in the same class. Another way of putting it is: VC $\leq$ INDSET $\leq$ VC makes VC and INDSET in the same class. Same for INDSET and CLIQUE. Therefore CLIQUE and VC are in the same class.

We also showed SAT $\leq$ CLIQUE, and CLIQUE $\leq$ NP, so now we have that all of these problems are in NP-complete.

More NP-complete problems

There are hundreds of NP-complete problems, but we're out of time to learn more. Here are some popularly referenced ones:

- HP: the Hamiltonian Path Problem. Is there a Hamiltonia Path in the graph? A Hamiltonian path is a path that visits each node exactly once.
  (I can't actually find a common abbreviation for this problem, so I'm using HP)
- TSP: travelling salesperson, decision version. Is there a path that visits each node exactly once that is shorter or equal to a target length?
  (normally TSP is stated as finding the shortest such circuit, but that version is FNP)
- The subset sum problem. Given a set of numbers and a target sum, is there any subset of the given set that has the target sum?
- The partition problem. Given a set of numbers, is it possible to partition it so that both partitions have the same sum?

Practice

- Reduce HP to TSP: it's surprisingly easy
- Reduce TSP to HP: harder, but if you are clever with the lengths you use for your edges and the target length for TSP, you can do it.
- Do we need to do anything else to prove NP-completeness of TSP and HP? If so, what would it take?
  This one is answered in the next slide.

Proving NP-completeness of TSP and HP

We haven't proved it's NP-complete.

We need to show:

- that TSP or HP is NP. Then we know they are both in NP.
- that an existing NP-complete problem can be reduced to one of them.

If we did this, both problems would be proven np-complete (which they are).

In practice, there is a clever reduction from 3SAT to HP that involves creating a series of diamonds of 4 vertices, one for each variable, that forces you to pick one way or the other depending on whether the variable is assigned true or not.

Beyond NP: NP-Hard

NP-complete is the hardest set that we've spent a lot of time on.

The problems in NP-complete are all $O(2^{p(n)})$, because all of them can be solved by considering every combination (for n inputs, there are 2^n combinations) and then some polynomial time work to verify each input.

Every NP-complete problem is in NP-hard, but NP-hard has some decision problems that are even slower than NP-complete.

Can anyone think of some?

NP-hard - NP-complete problems

Undecidable problems are harder than NP-complete (because there is no bound to how long they can take).

So the halting problem and equivalence problems are NP-hard and not NP-complete.

Determining if one piece of code is "maximally optimized" in general is also harder than NP-complete (it relies on equivalence).

There are also more conventional problems that are hard. For example: "is this the set of all permutations of n values" is $\Theta(n!)$

Below NP-complete but above P: NP-Inter

If P is not NP, then there are problems that are in NP, but not in P nor in NP-complete.

For now (as long as we don't know P = NP), we call this set NP-inter.

The most widely referenced NP-inter problem I'm aware of is integer factorization: given two integers, do they have a common factor greater than 1? This is exponential in the number of bits, but is much faster than $O(2^n)$

Another one is the discrete logarithm problem.

The non-P nature of these is important. Interestingly, many of these (but not all) are instances of a more fundamental problem called "abelian hidden subgroup" problems, which can be solved by quantum computers in polynomial time. Bad news for browser security!

Tested skills

This section will be tested by asking short problems that test the following:

- That you understand the definitions.
    - What is required for membership in P?
    - What is required for membership in NP?
    - What is required for membership in NP-inter?
    - What is required for membership in NP-hard?
    - What is required for membership in NP-complete?
- That you understand the reduction from any NP problem to SAT.
- That you understand SAT $\rightarrow$ 3SAT

More on the next slide...

Tested skills (2)

- That you understand SAT $\leq$ CLIQUE
- That you understand INDSET, CLIQUE, and VC, and the mutual reductions.
- That you are familiar with the complexity class of all the problems we've talked about in this lecture (i.e., if I say reduce a problem to discrete log, you know that means NP-inter)
- That you understand how to prove that problems belong to one category or another using reductions.

Quiz format

The quiz will be 4 questions. Each question will be short answer, and will require a brief explanation.

The 4 questions will be waited equally (25%). Self grading will be the same as always.

Example problems

1. Is the problem of determining if a list is sorted in NP? Give a 1 sentence explanation of your answer.
2. Do you think the problem of determining if a matrix is the product of two matrices is known to be in NP-complete? Give a 1 sentence explanation of your opinion.
3. Assuming P is not NP, are we aware of any problems in NP-Inter that are also in NP-complete? Explain your answer, and provide an example if your answer is "yes".
4. Suppose we learn that P = NP. Is NP-inter now empty?
5. Are there any members of NP-complete that are not in NP-hard? Explain your answer; provide an example if the answer is yes.

Example problems (2)

6. Suppose we are performing the sat reduction for a problem. $S_{t,i,a}$ is true and $S_{t+1,i+1,b}$ is true. Is is possible for both variables to be true? Explain your answer?
7. Suppose we have a graph with 100 nodes, and a minimum VC of 10 nodes. Does that say anything about the largest indset? If so, what is its size?
8. Suppose we just now found a polynomial time reduction of TSP $\leq_m^p$ List-is-sorted. Does this prove anything interesting we didn't already know? If so, what, and explain why? If not, explain why not.
9. Suppose we just now found find a polynomial time reduction of List-is-sorted $\leq$ TSP. Does this prove anything interesting we didn't already know? If so, what, and explain why? If not, explain why not.
10. Suppose TSP $\leq$ A $\leq$ B. If A is $\Omega(n!)$, which category can we assign B to?

Selected answers

For 1, yes, determining if a list is sorted is in P, and every problem in P is in NP.

For 3, no, if P $\neq$ NP, NP-inter and NP-complete are disjoint. If we showed an NP-inter problem were in NP-complete, it would imply that we reduced an NP-complete problem to an NP-inter problem in polynomial time, which would "merge" the two sets.

For 5, no, every member of NP-complete is in NP-hard by definition. NP-hard is the set of problems that are at least as hard as the problems in NP-complete.

For 8, yes, it would mean that P = NP, because we reduced an np-complete problem to a P problem in polynomial time.

For 9, no, we already knew the problem List-is-sorted can be reduced to TSP, because P $\leq$ NP $\leq$ SAT $\leq$ TSP.

More

These are just some of the types of questions I can ask.

I can also ask about cliques, 3SAT reductions, the reasons for complexity classes (instead of just relying on big-Θ), etc.

Anything in the slides is fair game.

However, now you can see what kinds of questions I am going to ask and roughly how long they will take.

I highly recommend letting an AI quiz you based on these slides. This is a truly good, healthy, and constructive use of AI.

Final video

I recommend this video to help you understand just how creative reductions can be.

The video shows a reduction from 3SAT into determining whether a Super Mario Bros. level is beatable.

This might help it click if you're struggling to understand reduction.

Appendix: Mermaid source

```
---
config:
      theme: redux
---
flowchart TD
  A --- B
  A --- C
  A --- D
  B --- C
```